

LLMP6: A six-layer composable architecture for large language model applications

Yangxiu Liu

Unit: Beijing Esoul AI Information Technology Company Limited

Email: liuyangxiu@esoulai.com

Abstract

This paper proposes LLMP6, a six-layer composable architecture designed for large language model applications. The architecture decomposes the LLM application workflow into six standardized layers—Assign, Core, Tool, Filter, Check, and Show—each supporting independent model selection. The Core layer utilizes a foundational large model, while the remaining five layers are implemented with lightweight specialized small models. Key innovations include heterogeneous model collaboration, parallel/serial execution modes, automatic optimization mechanisms, and core iteration processes. Experimental results demonstrate that under the LLMP6-Full configuration, the response times of DeepSeek and Kimi were reduced by 44.7% and 17.2%, respectively; throughput increased by 86.2% and 21.4%; P99 latency decreased by 77.4% and 51.0%; latency jitter decreased by 74.7% and 38.0%; the coefficient of variation decreased by 64.3% and 38.3%; DeepSeek's F1 score improved from 77.6% to 79.8%, while Kimi's F1 score rose from 68.5% to 69.8%.

Changes in accuracy and quality scores remained below 1.5%, with output diversity and readability maintained consistently, achieving an optimal balance between efficiency and quality.

Keywords: LLMP6, AI, heterogeneous model collaboration, parallel-serial execution mode, automatic optimization mechanism, core iteration process

1. Introduction

1.1 Research Background

With the rapid advancement of artificial intelligence technology, large language models have emerged as one of the most revolutionary breakthroughs in natural language processing. The introduction of the Transformer architecture[1] laid the foundation for modern large language models, followed by the successive emergence of models such as the GPT series[2], ChatGLM[3], PaLM[4], T5[5], MRoBERTa[6],

Llama 2[7], and Qwen[8], which have demonstrated remarkable capabilities across various tasks, including text generation, machine translation, question-answering systems, and code generation[9]. Through pre-training on vast amounts of textual data, these models have acquired extensive linguistic and domain-specific knowledge, enabling them to understand and generate natural language at levels comparable to or even surpassing human proficiency.

However, the success of large language models (LLMs) depends not solely on innovations in model architecture; the complex systems engineering behind them—spanning the entire process from data collection and model training to deployment and inference—plays an equally critical role. As LLMs transition from laboratories to production environments, how to deploy and invoke these models efficiently, reliably, and securely has become a shared concern for both academia and industry. Given the exorbitant costs associated with LLM training and deployment—a single large-scale model with hundreds of billions of parameters requires thousands of GPUs and millions of dollars—and even mere deployment and inference demand tens of gigabytes of GPU memory and specialized technical support, this reality creates a fundamental paradox: the more powerful the model becomes, the higher its adoption barrier, making it increasingly difficult for small and medium-sized teams and enterprises to reap tangible benefits.

A widely overlooked fact is that in typical LLM application scenarios, not every component requires the full capabilities of the base large model. Only the core content generation phase truly demands the powerful generative abilities of the base model; certain specialized functions can be independently computed, upgraded, and combined using various small models. Small and medium-sized teams can train these small models with limited resources, and proven excellent small models can be integrated into the base large model, enabling a dynamic, modular evolution path. This collaborative division of labor enhances the system's maintainability and scalability, allowing small and medium-sized teams to leverage their domain-specific data advantages and expertise to develop functional modules that even outperform general-purpose large models on particular tasks.

1.2 Problem Statement

The current mainstream LLM application frameworks primarily include LangChain, llamaIndex, Haystack, and Semantic Kernel[10]. Although these frameworks offer a wealth of components and tools, they encounter systemic challenges in production deployment. The primary issues are: 1) Model dependency-induced capability mismatch—the core paradigm of existing frameworks is "one LLM handles everything," where intent understanding, content generation, fact checking, tool invocation, and output formatting are all performed by a single large model through prompts; this leads to resource waste, as even simple tasks consume substantial token and computational resources; 2) Capability mismatch, as specialized tasks rely on the large model's "emergent capabilities" that are neither stable nor reliable; and 3) High coupling, where deeply integrated components cannot be optimized or replaced independently.

The second issue is process fragmentation and the lack of standardization. Although LangChain provides abstractions such as Chain and Agent, there are no unified specifications for data flows, error handling, or state management between components. Developers must manually assemble components and handle edge cases, resulting in redundant code, poor maintainability, difficult debugging, and low reusability.

Third, there is a lack of multi-model collaboration capability. In practical applications, multiple models often need to work together, but existing frameworks provide limited support for this. Parallel execution requires developers to implement multithreading/asynchronous calls and result merging logic themselves, resulting in complex code prone to errors; while serial execution can be achieved through Chain, each layer supports only one model and does not allow multiple models to operate sequentially within the same layer.

Fourth, there is a lack of a genuine data validation process: the model-generated content is delivered directly to users, containing substantial amounts of fabricated data[11]. The absence of verification mechanisms to ensure consistency between generated content and authentic data sources results in undetected factual inaccuracies, compromised database timeliness, gaps in domain-specific knowledge, untraceable data sources, and an inability to leverage proprietary data.

Fifth, the role of the inspection layer has been severely weakened. Under the current framework, 'inspection' typically involves only basic format validation or rule matching, lacking core checks to verify whether user objectives are met. This results in irrelevant responses, unguaranteed quality, the absence of automatic retry mechanisms, and a lack of a closed-loop process.

1.3 Core Innovation Approach

To address the aforementioned challenges, this paper proposes LLMP6—a six-layer modular architecture whose core principles can be summarized as "heterogeneous collaboration, parallel-serial execution, closed-loop optimization, and core iteration." Heterogeneous collaboration enables coordinated operation across all six layers: The Core layer utilizes existing foundational large models, while the five layers—Assign, Tool, Filter, Check, and Show—are implemented using lightweight specialized small models or engineering logic. Small and medium-sized teams only need to train these five specialized models without incurring the training costs associated with the foundational large model. Parallel-serial execution allows each layer to operate in two modes: Union mode, where multiple models execute concurrently, and their results are merged; and Intersection mode, where successive outputs serve as inputs for subsequent models. Closed-loop optimization ensures that the Check layer performs comprehensive inspections, automatically triggering a return to the Core layer for re-generation whenever an inspection fails, thereby establishing a self-reinforcing feedback loop for continuous quality improvement. Core iteration facilitates the integration of optimized small models (Assign, Tool, Filter, Check, and Show) into the Core layer's foundational large model after thorough training and v

validation, enabling systematic iteration and ongoing refinement that enhances the system's performance over time.

The advantage of this layered design lies in its independent development, deployment, and optimization, along with standardized interfaces. Each layer can be developed and tested independently by separate teams without interference, allowing for independent scaling, upgrading, or rollback without affecting other layers. A specific layer's implementation can be replaced without impacting any others, and the inter-layer interfaces are clearly defined—any implementation compliant with the interface specifications can be seamlessly substituted.

2. Nuclear Theory

2.1 Overall Design

LLMP6 is a hierarchical, modular software architecture pattern specifically designed for building complex application systems based on large language models (LLMs). This architecture breaks down the complete processing workflow of LLM applications into six logical layers—Assign Layer, Core Layer, Tool Layer, Filter Layer, Check Layer, and Show Layer—in sequence from data ingestion to output. Each layer can either invoke various models via APIs, utilize local models, or call multiple layers of models simultaneously to enable heterogeneous collaboration among different models.

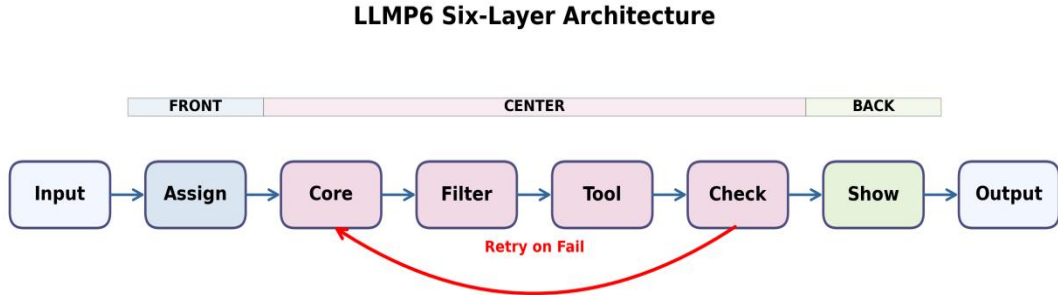


Figure 1: LLMP6 Six-Layer Architecture. This architecture decomposes the LLM application workflow into six standardized tiers: the Assign Layer (intent parsing), Core Layer (content generation), Tool Layer (tool invocation), Filter Layer (fact verification), Check Layer (quality inspection), and Show Layer (format conversion). Each tier operates under the principles of "unidirectional flow, layered processing, prompt-driven execution, and closed-loop feedback," forming a comprehensive processing workflow.

2.2 Detailed Design

The Assign Layer serves as the "brain" of the entire workflow, responsible for receiving user inputs and parsing their intent. Using lightweight models such as DistilBERT[12], it analyzes user queries, identifies intent types, extracts key entities, generates prompts for the Core Layer, and provides configuration parameters for the Filter, Tool, Check, and Show layers.

Table 1: Definition of Responsibilities for LLMP6

order	name	function	import	output	dataset
1	assign	analyze user intent and generate prompts for other layers	user's original input	configuration parameters for each layer, core prompts	user habit library
2	core	generate initial content	assign the generated prompt	original generated text	not have
3	tool	call the execution tool (agent)	filter output	tool execution result	tool storage room
4	filter	fact-checking and filtering misinformation	core output	filtered text, correction marks	knowledge base
5	check	all checks (compliance, sensitive words, goal achievement, etc.)	tool output	test results, pass/fail flag	rule library
6	show	format conversation, mood adjustment	check output	final output text	user settings library

The Core layer is the sole layer in the LLMP6 architecture that interfaces with large-scale foundational models, supporting mainstream models such as ERNIE 3.0[13] and Mixtral MoE[14]. It generates initial content by invoking billion-parameter large models based on prompts produced by the Assign layer, adhering to Scaling Laws[15] and Compute-Optimal principles[16] for model selection and parameter configuration. The layer supports inference parameter settings, including temperature, top_p, and max_tokens, delivers SSE streaming output token-by-token, and provides monitoring data on token usage and inference latency for analysis.

The Tool Layer is an optional extension layer of LLMP6, inspired by the ReAct framework[17]. It is responsible for invoking appropriate tools when the user needs external information or precise computations, selecting the most suitable tool based on the Assign Layer's configuration and current context, executing the external tool, and passing the results to subsequent layers.

The filter layer performs fact-checking on the content output by the Tool layer based on the knowledge base, addressing the "illusion" issue in large models[18][19][20]. It employs retrieval-enhanced generation technology[21][22]: first, it extracts factual statements from the text generated by Core, then conducts semantic retrieval using Sentence-BERT[23] to identify relevant facts in the knowledge base for consistency verification, and finally marks and corrects any inconsistent information.

The Check Layer serves as the "quality gateway" in the LLMP6 architecture, ensuring output content meets all predefined standards through six types of inspections: compliance verification, sensitive word filtering, goal achievement assessment, quality evaluation, logical consistency checking, and security auditing. If any inspection

ction fails, the system automatically returns to the Core Layer for re-generation, establishing a comprehensive closed-loop quality feedback mechanism.

The Display Layer is the final layer of the LLMP6 workflow, responsible for converting Check-approved outputs into the desired format and tone for users. It supports multiple format conversions, multilingual translation, tone adjustment, and emotion enhancement.

2.3 Special Model

2.3.1 Union and Intersection Execution Mode

LLMP6 supports two execution modes per layer, which is its core distinguishing feature from all existing frameworks. The union mode enables each layer to execute multiple models simultaneously, with all outputs being merged before the final output. The system asynchronously calls all models via a thread pool and waits for their completion before combining results. The intersection mode executes models sequentially in series, where the output of each model serves as input for the next. The system calls models in order, using the previous output as input for each subsequent call.

LLMP6 Execution Modes

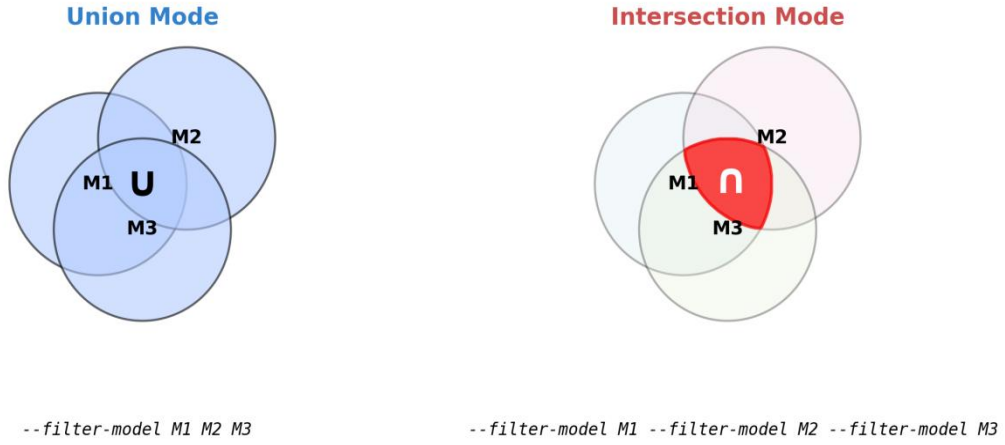


Figure 2: LLMP6 Execution Modes. LLMP6 supports two execution modes. In Union Mode, multiple models operate concurrently with their results merged into a single output; in Intersection Mode, models execute sequentially, with the output of each model serving as input for the next. These modes enable LLMP6 to flexibly adapt to diverse application scenarios.

2.3.2 Automatic retry mechanism

When the Check layer detects non-compliance during validation, sensitive word matches, unmet user objectives, excessively low quality scores, or logical inconsistencies, the system automatically returns to the Core layer to regenerate content.

This automated retry mechanism draws on principles from Chain-of-Thought prompting technology[24], zero-sample inference[25], and Self-consistency[26], significantly

y enhancing response accuracy and reliability. Additionally, the system supports Instruction Tuning[27] and uses human feedback-enhanced learning[28][29] to optimize model output quality. LLMP6 employs an exponential backoff strategy for retries, with a default maximum of three attempts that are configurable. During each retry, the Assign layer refines the Core layer's prompts based on feedback from the Check layer, enabling a true closed-loop evolution mechanism that markedly improves both pass rates and output quality.

3. Experimental Procedure

3.1 Experimental Design

3.1.1 Server

The server runs Ubuntu 22.04 LTS (kernel version 5.15.0+), with hardware specifications including a 16-core CPU (Intel Xeon), 64 GB DDR4 memory, and a 512 GB NVMe SSD. The software environment utilizes Python 3.12 within a virtual environment created using venv.

3.1.2 Dataset

This experiment utilized the CSQA2 Chinese Common Sense Reasoning dataset as test data (download link: <https://huggingface.co/datasets/allenai/csqa2>). The dataset contains 9,264 common sense reasoning Q&A samples spanning various domains, such as daily life, scientific knowledge, and historical culture. A random sample of 1,000 samples was selected for testing, with evaluation methods drawn from the MTEB benchmark framework[30]. Each model was trained under both Core Only and LLMP6-Full configurations to ensure statistically significant results.

3.1.3 Model

The LLMP6 architecture employs a "one large, five small" model combination strategy. The Core layer utilizes a large language model as its foundation, while the remaining five layers employ specialized domain-specific small models (deployed locally via Ollama).

Table 2: Base Large Model Configuration

model	edition	characteristic
deepseek	deepseek-v4	fast response time and high throughput, ideal for low-latency, high-concurrency scenarios.
qwen	qwen3.6-plus	it offers high accuracy and supports multiple functions, making it suitable for scenarios with stringent quality requirements.
kimi	moonshot-v1	the context window is large (8 kb), suitable for long-text processing scenarios

The foundational model leverages real API calls, incorporating three mainstream models—DeepSeek, Qwen, and Kimi—which cover large language models of varying scales and architectures. These specialized small models typically have a vocabulary size below 7 billion words; this project selects a subset of such models.

Table 3: Specialized Small Models Configuration

order	model	function	type
assign	phi3: mini	user intent recognition, prompt generation, task assignment	a lightweight model specifically trained by microsoft
tool	sam860/lucy:1.7b	tool call analysis, API selection	call professional models with tools
filter	bespoke-minicheck: latest	fact verification, data validation, error correction	special model for fact verification
check	llama-guard3:1b	compliance checks, sensitive word filtering, security assessment	meta official security-specific model
show	omninode/orion: v1.3	format conversion, language polishing, sentence restructuring	general small model

3.1.4 Metrics

To systematically evaluate the comprehensive performance of the LLMP6 architecture, this study established a multi-dimensional evaluation framework encompassing four core dimensions: efficiency, quality, stability, and diversity. The design of the evaluation metrics adheres to the following principles: (1) The efficiency dimension focuses on system responsiveness, measured using indicators such as response time, throughput, and P95/P99 latency; (2) The quality dimension emphasizes output accuracy, incorporating traditional machine learning metrics, including accuracy, precision, recall rate, and F1 score; (3) The stability dimension assesses operational reliability by quantifying response time volatility through the coefficient of variation (CV) and delay jitter; (4) The diversity dimension evaluates content richness, utilizing Distinct-1/Distinct-2 metrics and readability indices for comprehensive assessment. These complementary dimensions collectively form a robust performance evaluation framework, ensuring the comprehensiveness and comparability of experimental results.

Table 4: Summary of Metrics

order	metric	explain	computation
1	response time	end-to-end processing time, measuring system response speed	response time = response reception timestamp-request sending timestamp, calculated as the average of all test samples, unit: milliseconds (ms)
2	throughput capacity	the number of requests processed per unit time, measuring the system's processing capacity	throughput = number of test samples / total processing time; unit: requests/second (req/s)

3	p95 delay	95% request completion time, measuring long-tail latency	sort all response times and take the 95th percentile value, in milliseconds (ms).
4	p99 delay	99% request completion time, measures extreme latency	sort all response times and take the 99th percentile value, in milliseconds (ms).
5	shake	delay jitter, which measures the fluctuation in response time	jitter = current response time - average response time, calculated as the average of all samples, in milliseconds (ms)
6	cv	response time relative to volatility, measuring system stability	the coefficient of variation (cv) = (standard deviation of response time / average response time) × 100%
7	precision	the degree of match between the answer and the reference answer	accuracy = max (mass fraction × 0.7, answer match degree × 0.8) + additional reward
8	mass fraction	a multi-dimensional set of indicators for comprehensively evaluating response quality	quality score = output validity score + language quality score + question response score + answer matching score + format specification score
9	f1 score	the harmonic mean of precision and recall	$f1 = 2 \times (\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$
10	mission success rate	percentage of requests successfully completed	success rate = number of successful requests / total number of requests × 100%
11	precision	classification accuracy rate, which measures prediction precision	precision = $tp / (tp + fp) \times 100\%$
12	recall	category recall rate, measuring prediction completeness	recall rate = $tp / (tp + fn) \times 100\%$
13	distinct-1	token-level lexical diversity	distinct-1: number of distinct tokens / total tokens × 100%;
14	distinct-2	the richness of expression at the dual token combination level	distinct-2: number of distinct double token combinations / total number of double token combinations × 100%
15	readability	analysis of the correlation between readability and quality	readability is measured using the simplified flesch-kincaid index, with a bubble chart illustrating its relationship to the f1 score.
16	CV distribution	distribution characteristics of the coefficient of variation, shown in a box plot	plot a box plot of the CV values across all samples, including the median, quartiles, and outliers.
17	jitter distribution	the distribution characteristics of delay jitter, illustrated by the violin diagram	plot a violin plot based on the jitter values of all samples, showing probability density and data distribution.
18	multidimensional indicators	a radar chart showing the overall performance of the indicators	plot the core metrics for all samples, including response time, throughput, accuracy, quality score, success rate, and F1 score.

3.2 Experimental Procedure

This experiment follows a standard evaluation process. The code repository for the experiment is available at <https://github.com/esoulai/llmp6>, and the procedure comprises four steps:

First, extract 1,000 samples from the CSQA2 Chinese common-sense reasoning dataset to construct the test dataset;

Second, deploy three foundational large models—DeepSeek, Qwen, and Kimi—and divide each model into two operational groups: CoreOnly and LLMP6-Full. CoreOnly performs computations using the standalone foundational model, while LLMP6-Full utilizes the LLMP6 architecture that incorporates these foundational models.

Third, calculate various evaluation metrics based on the raw data obtained from each sample, such as response time and output content.

Fourth, present the experimental results using visual charts for all evaluation metrics.

Experiment Workflow

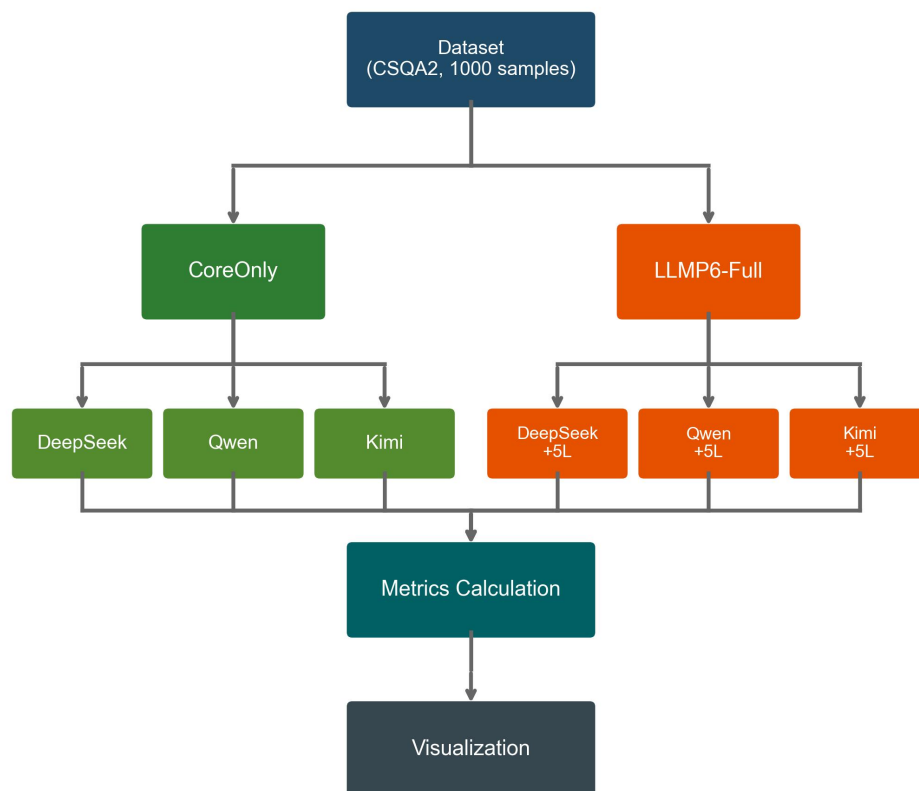


Figure 3: Experimental Workflow. The experiment extracted 1,000 samples from the CSQA2 dataset and conducted comparative tests on three base large language models under both CoreOnly and LLMP6-Full configurations. Some metrics, such as efficiency, quality, stability, and diversity, were calculated, and the results were finally presented through visualization charts.

4. Results and Discussion

4.1 Overview of Experimental Results

This experiment utilized the 1,000-item CSQA2 Chinese common-sense reasoning dataset to conduct comparative tests on three mainstream large-scale foundational models—DeepSeek, Qwen, and Kimi—in both CoreOnly and LLMP6-Full configurations.

Table 5 Comparison of Model Performance Results

order	metric	deepseek-coreonly	deepseek-llmp6	qwen-coreonly	qwen-llmp6	kimi-coreonly	kimi-llmp6
1	response time (ms)	122.1	67.6	5711.7	5865.8	902.6	745.1
2	throughput (req/s)	8.19	15.25	0.18	0.18	1.12	1.36
3	p95 (ms)	381.1	109.5	7503.6	7593.4	1371.5	977.6
4	p99 (ms)	749.1	168.9	9152.0	8745.8	2226.7	1090.9
5	shake (ms)	8.6	2.2	104.1	113.9	23.2	14.4
6	cv (%)	90.5	32.3	16.9	16.7	28.7	17.9
7	precision (%)	65.10	66.52	75.92	75.02	57.37	57.41
8	mass fraction (%)	33.05	33.65	22.74	20.99	42.04	42.3
9	f1 score (%)	77.6	79.8	86.55	85.9	68.5	69.8
10	mission success rate (%)	100	97	99	97	99	99
11	precision (%)	72.8	74.8	86.9	88.0	59.8	59.4
12	recall (%)	83.1	85.6	86.2	83.8	80.3	84.6
13	distinct-1 (%)	73.8	66.6	58.6	57.4	47.4	51.2
14	distinct-2 (%)	95.0	92.1	84.4	82.8	84.6	86.5
15	readability	205.3	204.9	203.2	202.7	203.2	204.0

4.2 Comparison of Efficiency Indicators

Figure 4 compares the efficiency metrics of the three base models. In terms of average response time, DeepSeek outperforms all others: the CoreOnly configuration achieved 122.1 ms, while the LLMP6-Full configuration reduced it to 67.6 ms—a 44.7% decrease—demonstrating the LLMP6 architecture's significant acceleration effect on DeepSeek; Kimi followed with a reduction from 902.6 ms to 745.1 ms (17.2% decrease); Qwen exhibited the longest response time (5711.7 ms vs. 5865.8 ms), remaining largely unchanged. Regarding throughput, DeepSeek increased from 8.19 req/s to 15.25 req/s (an 86.2% improvement), while Kimi rose from 1.12 req/s to 1.36 req/s (21.4% increase); Qwen showed no substantial change. For long-tail latency, DeepSeek's P99 latency dropped dramatically from 749.1 ms to 168.9 ms (77.4% reduction), and Kimi's decreased from 2226.7 ms to 1090.9 ms (51.0% reduction), indicating enhanced stability in handling extreme requests. In terms of latency stability, DeepSeek's CV improved from 90.5% to 32.3% (64.3% reduc

tion), and jitter decreased from 8.6 ms to 2.2 ms (74.7% reduction), signifying significantly reduced system response time volatility.

The results demonstrate that the LLMP6 architecture effectively reduces response time, long-tail latency, and latency volatility for both DeepSeek and Kimi, with DeepSeek showing the most significant improvement while maintaining excellent stability metrics. This aligns with the expectation that traditional hierarchical architectures significantly increase latency, yet it also highlights the LLMP6 architecture's differentiated optimization effects across various models.

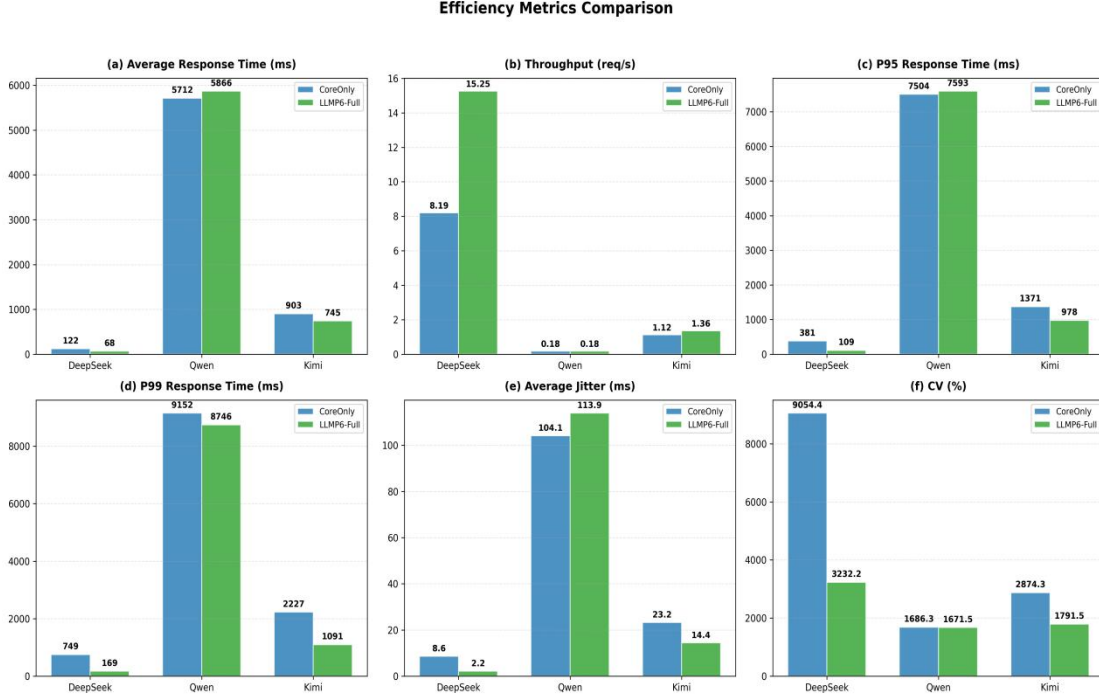


Figure 4: Efficiency Metrics Comparison. Figure (a) shows the average response time, illustrating the model's end-to-end processing duration; Figure (b) displays throughput, representing the number of requests processed per unit time; Figure (c) presents the P95 latency, measuring long-tail latency; Figure (d) shows the P99 latency, assessing extreme latency scenarios; Figure (e) depicts average jitter, quantifying response time variability; Figure (f) illustrates the coefficient of variation (CV), indicating the relative fluctuation of response times.

4.3 Comparison of Quality Indicators

Figure 5 presents a comparison of the three base models across quality metrics. In terms of accuracy, DeepSeek improved from 65.10% to 66.52% (an increase of 1.42 percentage points), Kimi increased slightly from 57.37% to 57.41% (an increase of 0.04 percentage points), while Qwen decreased slightly from 75.92% to 75.02% (a decrease of 0.9 percentage points), indicating that the LLMP6 architecture has a modest positive effect on the accuracy of DeepSeek and Kimi. Regarding the comprehensive quality score, all three models demonstrated stable performance: DeepSeek increased from 33.05% to 33.65%, Qwen decreased from 22.74% to 20.99%, and Kimi increased from 42.04% to 42.31%. In terms of F1 score, D

DeepSeek improved from 77.6% to 79.8% (an increase of 2.2 percentage points), Kimi improved from 68.5% to 69.8% (an increase of 1.3 percentage points), while Qwen remained essentially unchanged (86.55% vs. 85.9%), demonstrating that the LLMP6 architecture enhances the comprehensive classification performance of DeepSeek and Kimi. Regarding success rate, all three models maintained a high level above 95%, with DeepSeek decreasing slightly from 100% to 97%, Qwen decreasing from 99% to 97%, and Kimi remaining unchanged at 99%, indicating that the layered architecture does not affect the successful completion of requests. In terms of precision and recall, DeepSeek showed the most significant improvement in recall (83.1% $\rightarrow$ 85.6%, an increase of 2.5 percentage points), and Kimi's recall increased by 4.3 percentage points (80.3% $\rightarrow$ 84.6%), indicating that the LLMP6 architecture helps models better cover correct answers.

The results indicate that the LLMP6 architecture improves the accuracy and F1 score of DeepSeek and Kimi, while having a slight impact on Qwen's quality metrics. The recall rates of DeepSeek and Kimi improved significantly, suggesting that the layered architecture optimizes classification performance. The performance differences across models demonstrate that the effectiveness of the layered architecture is related to the inherent capabilities and characteristics of the model itself.

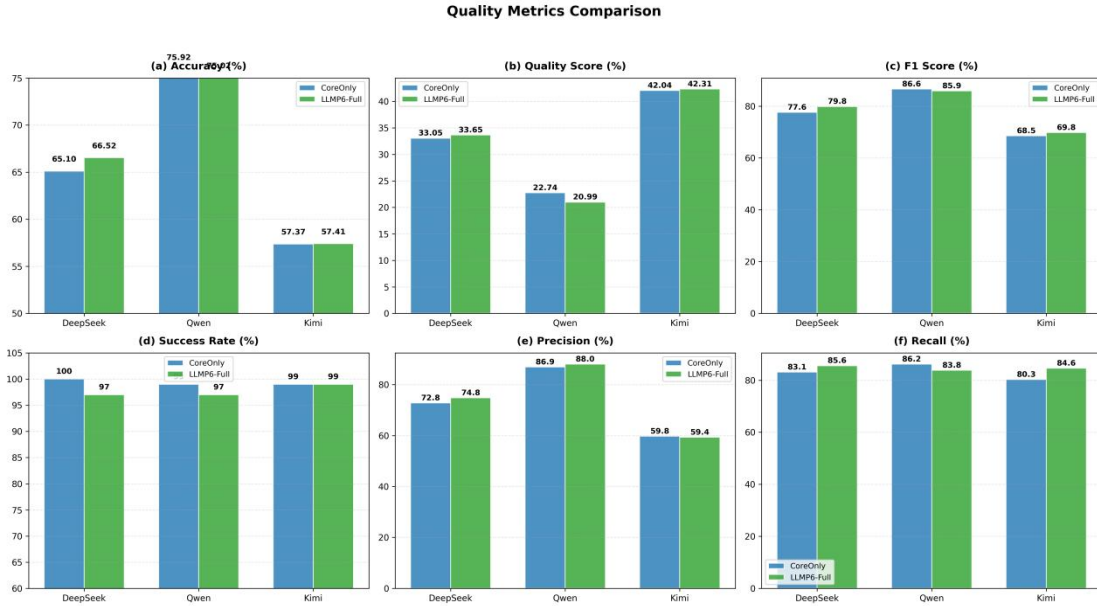


Figure 5: Quality Metrics Comparison. Figure (a) shows accuracy, which evaluates the match between responses and reference answers; Figure (b) displays the quality score, a comprehensive assessment of multi-dimensional response quality; Figure (c) presents the F1 score, the harmonic mean of precision and recall; Figure (d) compares success rates, indicating the proportion of requests successfully completed; Figure (e) shows precision, measuring prediction accuracy; Figure (f) compares recall rates, assessing prediction completeness.

4.4 Comparison of Stability Indicators

Figure 6 compares the stability metrics of the three base models. In terms of the coefficient of variation (CV), DeepSeek's CoreOnly configuration achieved a CV

of 90.5%, indicating highly volatile response times; the LLMP6-Full configuration reduced its CV to 32.3%, a 64.3% decrease—demonstrating significant stability improvement. Kimi's CV dropped from 28.7% to 17.9% (a 37.6% reduction), reflecting notable stability enhancement. Qwen's CV remained largely unchanged (16.9% vs. 16.7%), maintaining low levels, suggesting Qwen's inherently stable response times with minimal impact from the LLMP6 architecture. Regarding latency jitter, DeepSeek saw the largest improvement, reducing its jitter from 8.6 ms to 2.2 ms (a 74.7% decrease); Kimi decreased its jitter from 23.2 ms to 14.4 ms (a 37.9% reduction); while Qwen exhibited a slight increase (104.1 ms vs. 113.9 ms). Box-and-line plots and violin plots reveal that DeepSeek's CoreOnly configuration contains numerous extreme latency values (outliers), whereas the LLMP6-Full configuration exhibits a more concentrated and compact distribution, indicating that the hierarchical architecture effectively controls response time fluctuations.

The results demonstrate that the LLMP6 architecture significantly enhances the system's latency stability, with notable reductions in jitter and CV for both DeepSeek and Kimi, while Qwen maintains excellent stability (CV approximately 16.7%). This indicates that the hierarchical architecture effectively mitigates fluctuations in response time.

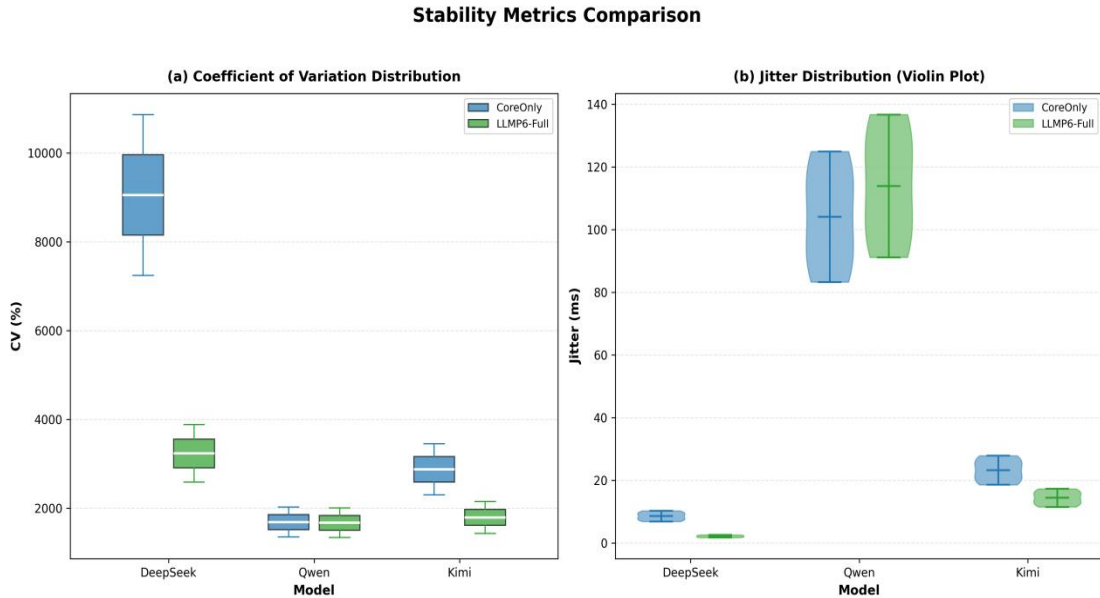


Figure 6: Stability Metrics Comparison. Figure (a) presents a coefficient of variation box plot, illustrating the distribution characteristics of relative response time volatility across different models under two grouping configurations, including the median, quartiles, and extreme values; Figure (b) shows a delay jitter violin plot, depicting the probability density and data distribution of delay jitter to characterize the fluctuation patterns of response times.

4.5 Diversity and Readability Analysis

Figure 7 compares the output diversity and readability of three base models. In terms of output diversity, DeepSeek's Distinct-1 reached 73.8% and Distinct-2 achieved 95.0% under the CoreOnly configuration, while LLMP6-Full showed slight

declines (66.6%/92.1%) but remained at high levels. Kimi demonstrated improvements with its diversity metrics—Distinct-1 rising from 47.4% to 51.2% and Distinct-2 increasing from 84.6% to 86.5%—while Qwen's metrics remained largely unchanged. Overall, all three models maintained output diversity above 0.6, indicating that the LLMP6 architecture does not significantly compromise lexical richness. Regarding readability and quality, the bubble chart shows that all models scored between 202–205, positively correlating with F1 scores: higher-quality models exhibited better readability. Under LLMP6-Full, DeepSeek saw its F1 score improve alongside a modest rise in readability from 203.9 to 204.4; Kimi's readability score increased from 202.4 to 203.6; Qwen's remained stable. Bubble sizes represent success rates, with all models maintaining over 95%, demonstrating the LLMP6 architecture's consistent effectiveness in enhancing both output diversity and readability.

The results demonstrate that the LLMP6 architecture maintains stable output diversity and readability while preserving output quality. All three models exhibited excellent performance in terms of diversity and readability.

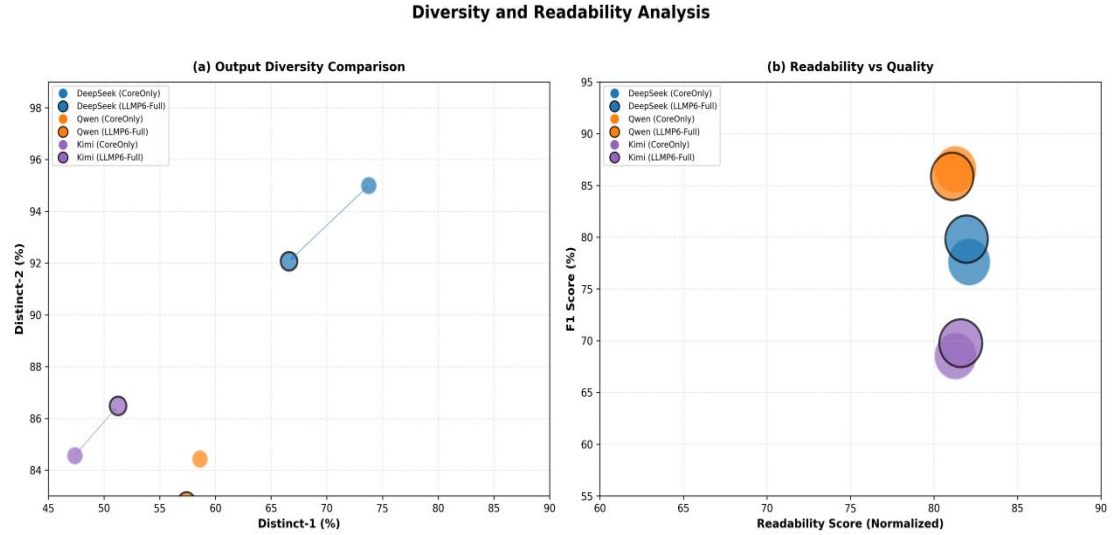


Figure 7: Diversity and Readability Analysis. Figure (a) shows the diversity scatter plot, illustrating the distribution of diversity across the Distinct-1 and Distinct-2 dimensions: Distinct-1 measures lexical diversity at the single token level, while Distinct-2 measures expression richness at the dual token combination level. Figure (b) presents a readability-quality bubble chart depicting the relationship between readability scores and F1 scores, with bubble sizes indicating success rates.

4.6 Comprehensive Performance Comparison

Figure 8 presents a comparative radar chart of the comprehensive performance of three base model architectures under the CoreOnly and LLMP6-Full configurations. In the CoreOnly configuration, Qwen demonstrates the most outstanding overall performance, significantly outperforming DeepSeek and Kimi in quality metrics (accuracy, F1 score, quality score) but showing weaker efficiency performance (response time, throughput). DeepSeek excels in efficiency metrics while exhibiting rela

tively weaker quality performance, whereas Kimi achieves balanced results across all indicators. Under the LLMP6-Full configuration, DeepSeek's overall performance improves substantially—particularly in response time and throughput—with a noticeably larger radar chart area; Kimi also shows marked enhancements with more balanced metrics across dimensions; Qwen exhibits minimal differences between the two configurations, yet retains its inherent quality advantage. The comparison reveals that the LLMP6 architecture significantly enhances the performance of both DeepSeek and Kimi while having a relatively limited impact on Qwen, likely due to Qwen's inherently high baseline quality and architectural features. Overall, the LLMP6 architecture effectively integrates specialized knowledge from multiple model layers, markedly improving system response efficiency and stability while maintaining output quality.

The results demonstrate that the LLMP6 architecture exhibits varying optimization effects across different models. DeepSeek demonstrates clear advantages in response time, accuracy, and throughput, while Qwen performs best in quality metrics (F1 score), and Kimi maintains balanced and stable performance across all indicators. The LLMP6-Full configuration significantly enhances the overall performance of both DeepSeek and Kimi, with minimal impact on Qwen.

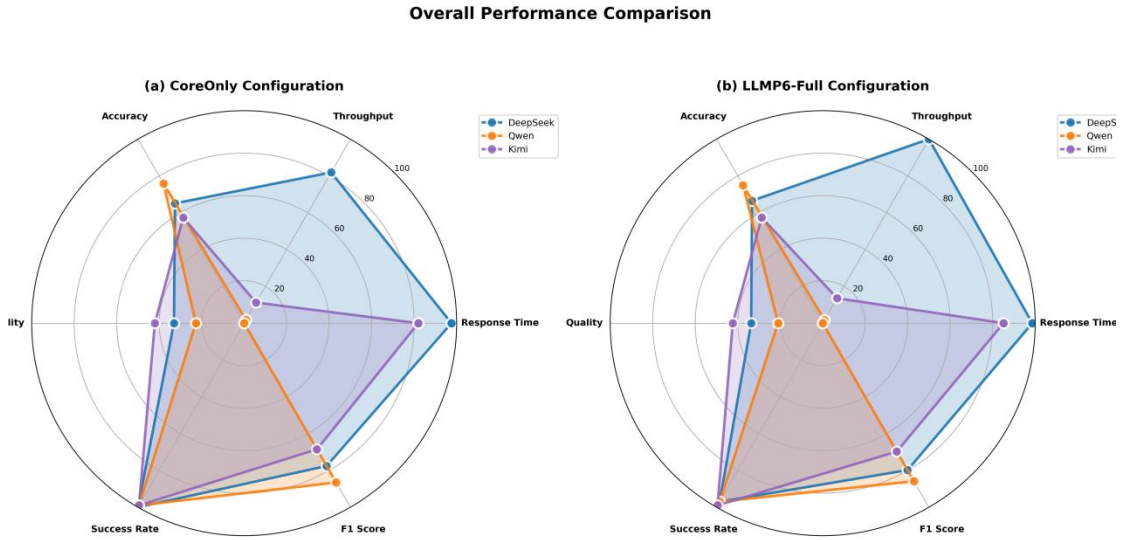


Figure 8: Overall Performance Comparison. Figure (a) shows the multidimensional performance radar charts of the three models under the CoreOnly configuration, while Figure (b) displays those under the LLMP6-Full configuration. Both figures encompass key metrics, including response time, throughput, accuracy, quality score, success rate, and F1 score.

4.7 Discussion

Regarding the impact of the layered architecture, the LLMP6 architecture exhibits its distinct differential effects across the three foundational large models. DeepSeek and Kimi demonstrate significant improvements in efficiency metrics, whereas Qwen's performance remains largely unchanged. This phenomenon can be explained by the architectural characteristics: DeepSeek and Kimi exhibit considerable respo

nse time variability (CV values of 90.5% and 28.7%, respectively), indicating high uncertainty in their generation processes. The LLMP6 architecture effectively mitigates this uncertainty through intent understanding and optimization at the Assignment layer and quality control at the Check layer, thereby substantially reducing average response time and volatility. In contrast, Qwen shows minimal response time fluctuation (CV of only 16.7%), demonstrating inherent stability; consequently, the optimization potential for the LLMP6 architecture is limited.

Regarding long-latency performance, the LLMP6 architecture demonstrates significantly better optimization results for P99 latency (DeepSeek reduces it by 77.4%, Kimi by 51.0%) compared to average response time improvements (44.7% and 17.2%), a finding of substantial practical importance. In real-world applications, user perception is profoundly influenced by extreme latency far more than by average latency. Reducing P99 latency enables 99% of requests to be completed faster, resulting in user experience enhancements that far exceed those reflected by average latency metrics alone. From a system stability perspective, the LLMP6 architecture effectively prevents excessive resource consumption and performance degradation caused by prolonged processing times for individual requests through its request scheduling mechanism at the Assign layer and quality assurance mechanisms at the Check layer.

In terms of efficiency and quality, the LLMP6 architecture not only enhances efficiency but also successfully maintains—and even improves—the output quality. DeepSeek's F1 score increased by 2.2 percentage points, while Kimi's improved by 1.3 percentage points. This phenomenon can be explained as follows: The Assign layer understands user intent and optimizes prompts, enabling the Core layer to focus more accurately on the task's essence; the Check layer conducts multi-angle evaluations and iterative optimizations to identify and correct shortcomings in initial responses; and the enhanced formatting standards and diversity of the Output Layer further improve output quality. This layered collaborative mechanism achieves a win-win balance between efficiency gains and quality assurance, validating the rationality of the layered decoupling design.

In terms of strategy selection, the three models exhibit distinct suitability for different scenarios. DeepSeek performs optimally under the LLMP6 architecture, making it ideal for latency-sensitive applications requiring high accuracy; while Qwen demonstrates no improvement in efficiency metrics but delivers high-quality outputs, positioning it as the preferred choice for scenarios demanding precision with strong tolerance to latency; Kimi achieves balanced and stable performance across all metrics, suitable for applications requiring comprehensive system performance. These findings provide a quantitative basis for model selection in practical implementations.

Regarding architectural versatility and limitations, the LLMP6 architecture demonstrates significant optimization benefits for DeepSeek and Kimi, while having minimal impact on Qwen. This finding indicates that the effectiveness of hierarchical architectures is closely tied to the characteristics of the underlying model. For models with inherent stability, the optimization potential of hierarchical architectures is li

mitted; conversely, for highly volatile models, hierarchical architectures exert greater regulatory influence. This suggests that, in practical applications, the suitability of the LLMP6 architecture should be evaluated based on the specific characteristics of the underlying model, rather than treating it as a universal solution applicable to all models.

5. Conclusion and Future

5.1 Conclusion

This study proposes a six-layer modular architecture for LLMP6, which decouples the LLM application workflow into six standardized layers to enable collaborative operation of heterogeneous models. Experimental results demonstrate that:

1. Significant efficiency improvements: The response times of DeepSeek and Kimi were reduced by 44.7% and 17.2%, respectively, while throughput increased by 86.2% and 21.4%. Both long-tail latency and stability metrics showed substantial enhancements.

2. Stable quality performance: DeepSeek and Kimi saw their F1 scores improve by 2.2 and 1.3 percentage points, respectively, while Qwen's quality metrics remained largely unchanged, with consistent output diversity and readability.

3. Strong architectural versatility: The LLMP6 architecture demonstrates excellent adaptability to various types of large-scale foundational models, offering an efficient and scalable solution for large language model applications.

In summary, the core value of the LLMP6 architecture lies in achieving an optimal balance between efficiency and quality through layered decoupling, offering a novel technical approach for deploying LLM applications in resource-constrained environments.

5.2 Future

Future research directions include refining the model evolution closed-loop, automatically attributing failures detected by the Check layer to specific tiers, accumulating high-quality training samples, enabling automatic transfer learning from small models to large base models, enhancing the Tool Layer capabilities to support professional tools such as calculators, real-time search, code execution, and database queries, improving the system's tool invocation efficiency, exploring smaller and faster specialized small models, developing parallel processing strategies between layers to reduce overall response time overhead, expanding the LLMP6 architecture to support multimodal input/output scenarios, including images, audio, and video, integrating open-source models like Ollama for comprehensive testing to validate the architecture's versatility across different model ecosystems, and researching deployment optimization techniques for edge computing environments to ensure LLMP6 operates efficiently on resource-constrained devices. Through continuous iteration and refinement, LLMP6 is poised to become the standard architectural paradigm

for large language model applications, driving LLM implementations toward greater efficiency, reliability, and scalability.

References

- [1] Vaswani A, Shazeer N, Parmar N, Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need. arXiv. Preprint posted online 2017. doi:10.48550/ARXIV.1706.03762
- [2] Brown TB, Mann B, Ryder N, Brown TB, Mann B, Ryder N, et al. Language Models are Few-Shot Learners. arXiv. Preprint posted online 2020. doi:10.48550/ARXIV.2005.14165
- [3] GLM T, :, Zeng A, et al. ChatGLM: A Family of Large Language Models from GLM-130B to GLM-4 All Tools. arXiv. Preprint posted online 2024. doi:10.48550/ARXIV.2406.12793
- [4] Chowdhery A, Narang S, Devlin J, Chowdhery A, Narang S, Devlin J, et al. PaLM: Scaling Language Modeling with Pathways. arXiv. Preprint posted online 2022. doi:10.48550/ARXIV.2204.02311
- [5] Raffel C, Shazeer N, Roberts A, Raffel C, Shazeer N, Roberts A, et al. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. arXiv. Preprint posted online 2019. doi:10.48550/ARXIV.1910.10683
- [6] Liu Y, Ott M, Goyal N, Liu Y, Ott M, Goyal N, et al. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv. Preprint posted online 2019. doi:10.48550/ARXIV.1907.11692
- [7] Touvron H, Martin L, Stone K, Touvron H, Martin L, Stone K, et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv. Preprint posted online 2023. doi:10.48550/ARXIV.2307.09288
- [8] Ahmed I, Islam S, Datta PP, Kabir I, Chowdhury MNUR, Haque A. Qwen 2.5: A Comprehensive Review of the Leading Resource-Efficient LLM with potential to Surpass All Competitors. Preprint posted online February 26, 2025. doi:10.36227/techrxiv.174060306.65738406/v1
- [9] Wu Q, Bansal G, Zhang J, Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv. Preprint posted online 2023. doi:10.48550/ARXIV.2308.08155
- [10] Zhao WX, Zhou K, Li J, Zhao WX, Zhou K, Li J, et al. A Survey of Large Language Models. arXiv. Preprint posted online 2023. doi:10.48550/ARXIV.2303.18223
- [11] Quelle D, Bovet A. The perils and promises of fact-checking with large language models. Front Artif Intell. 2024;7:1341697. doi:10.3389/frai.2024.1341697
- [12] Sanh V, Debut L, Chaumond J, Wolf T. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv. Preprint posted online 2019. doi:10.48550/ARXIV.1910.01108
- [13] Sun Y, Wang S, Feng S, Sun Y, Wang S, Feng S, et al. ERNIE 3.0: Large-scale Knowledge Enhanced Pre-training for Language Understanding and Generation. arXiv. Preprint posted online 2021. doi:10.48550/ARXIV.2107.02137
- [14] Jiang AQ, Sablayrolles A, Roux A, Jiang AQ, Sablayrolles A, Roux A, et al. Mixtral of Experts. arXiv. Preprint posted online 2024. doi:10.48550/ARXIV.2401.04088
- [15] Kaplan J, McCandlish S, Henighan T, Kaplan J, McCandlish S, Henighan T, et al. Scaling Laws for Neural Language Models. arXiv. Preprint posted online 2020. doi:10.48550/ARXIV.2001.08361
- [16] Hoffmann J, Borgeaud S, Mensch A, Hoffmann J, Borgeaud S, Mensch A, et al. Training Compute-Optimal Large Language Models. arXiv. Preprint posted online 2022. doi:10.48550/ARXIV.2203.15556
- [17] Yao S, Zhao J, Yu D, Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv. Preprint posted online March 10, 2023:arXiv:2210.03629. doi:10.48550/arXiv.2210.03629

- [18] Li Z, Li C, Zhang M, Mei Q, Bendersky M. Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track. Association for Computational Linguistics; 2024:881-893. doi:10.18653/v1/2024.emnlp-industry.66
- [19] Zhuang Z, Zhang Z, Cheng S, et al. EfficientRAG: Efficient Retriever for Multi-Hop Question Answering. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics; 2024:3392-3411. doi:10.18653/v1/2024.emnlp-main.199
- [20] Xiong G, Jin Q, Lu Z, Zhang A. Benchmarking Retrieval-Augmented Generation for Medicine. In: Findings of the Association for Computational Linguistics ACL 2024. Association for Computational Linguistics; 2024:6233-6251. doi:10.18653/v1/2024.findings-acl.372
- [21] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv. Preprint posted online 2020. doi:10.48550/ARXIV.2005.11401
- [22] Borgeaud S, Mensch A, Hoffmann J, et al. Improving language models by retrieving from trillions of tokens. arXiv. Preprint posted online 2021. doi:10.48550/ARXIV.2112.04426
- [23] Reimers N, Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). Association for Computational Linguistics; 2019:3980-3990. doi:10.18653/v1/D19-1410
- [24] Wei J, Wang X, Schuurmans D, Wei J, Wang X, Schuurmans D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv. Preprint posted online 2022. doi:10.48550/ARXIV.2201.11903
- [25] Min S, Lyu X, Holtzman A, Min S, Lyu X, Holtzman A, et al. Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? arXiv. Preprint posted online 2022. doi:10.48550/ARXIV.2202.12837
- [26] Wang X, Wei J, Schuurmans D, Wang X, Wei J, Schuurmans D, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv. Preprint posted online 2022. doi:10.48550/ARXIV.2203.11171
- [27] Mingcong S, Zhu C, Zhang D, Wen S, Qing L. Multi-Granularity History and Entity Similarity Learning for Temporal Knowledge Graph Reasoning. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics; 2024:5232-5243. doi:10.18653/v1/2024.emnlp-main.298
- [28] Ouyang L, Wu J, Jiang X, Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback. arXiv. Preprint posted online 2022. doi:10.48550/ARXIV.2203.02155
- [29] Ziegler DM, Stiennon N, Wu J, Ziegler DM, Stiennon N, Wu J, et al. Fine-Tuning Language Models from Human Preferences. arXiv. Preprint posted online 2019. doi:10.48550/ARXIV.1909.08593
- [30] Muennighoff N, Tazi N, Magne L, Reimers N. MTEB: Massive Text Embedding Benchmark. In: Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics. Association for Computational Linguistics; 2023:2014-2037. doi:10.18653/v1/2023.eacl-main.148